

Exploration of Rewards Shaping and Immitation Learning in Chess

24.01.2026

Jakob Lambert-Hartmann

Contents

Introduction	2
Methods	2
Architecture	2
Behavior Cloning	3
Puzzle Data	4
Grandmaster Data	4
Rewards Shaping	5
Findings	5
Bibliography	5

Introduction

Reinforcement learning (RL) is a field of machine learning concerned with decision-making. It is used in scenarios where an intelligent agent (e.g. chess player) takes actions in an environment (e.g. chess game) in order to maximize a reward (e.g. win). The behavior of an agent is called a policy. If rewards are sparse, it is difficult for an agent to learn a good policy. Winning a game of chess for instance is a sparse reward. It is difficult for an agent to learn chess with win as only reward, as it is very unlikely for an agent to randomly win a game. To help agents learn policies faster reward shaping introduces further rewards that align with the original goal and help to nudge the model in the right direction. Reward shaping however introduces a new set of problems. If the newly introduced rewards are not aligning with the ultimate goal for instance, the model may choose to prioritize those and ignore the real goal (reward hacking). [1]

Imitation learning is a paradigm in reinforcement learning, in which an agent learns a policy through a set of expert demonstrations, rather than an explicit reward function. Behavior cloning is a specific imitation learning technique, which utilizes supervised learning to learn this policy. However, expert demonstrations used for behavior cloning are often not uniformly sampled from the state space leading to poor performance. [2]

One Idea for mitigating the drawbacks of both reward based reinforcement learning and behavior cloning would be to combine the two approaches. Behavior cloning as a pretraining could reduce the need for reward shaping, while finetuning with reward based reinforcement learning could iron out the sampling bias from the expert demonstrations. In this document we investigate the interaction between pretraining with behavior cloning and finetuning with reward shaping using different sets of expert demonstrations and reward functions. We will apply this to the domain of chess, as available data is well split into different sampling biases.

Methods

Architecture

While the hidden layers of the agent differ, they share the same input and output representation. The network **input** contains the current position, which consists of a piece encoding (12 pieces: pawn, rook, knight, bishop, queen, king for both black and white) for each square on the chess board (8×8), resulting in an input tensor $b \in \mathbb{R}^{8 \times 8 \times 12}$.

The **output** consists of two tensors representing from which square to move m_{from} to which square m_{to} . A joint probability is created by computing from-to logits $M = m_{\text{from}} m_{\text{to}}^T$, masking out illegal moves (see Figure 1), and computing the softmax $\pi(b) = \sigma(\text{mask}(m_{\text{from}} m_{\text{to}}^T))$.

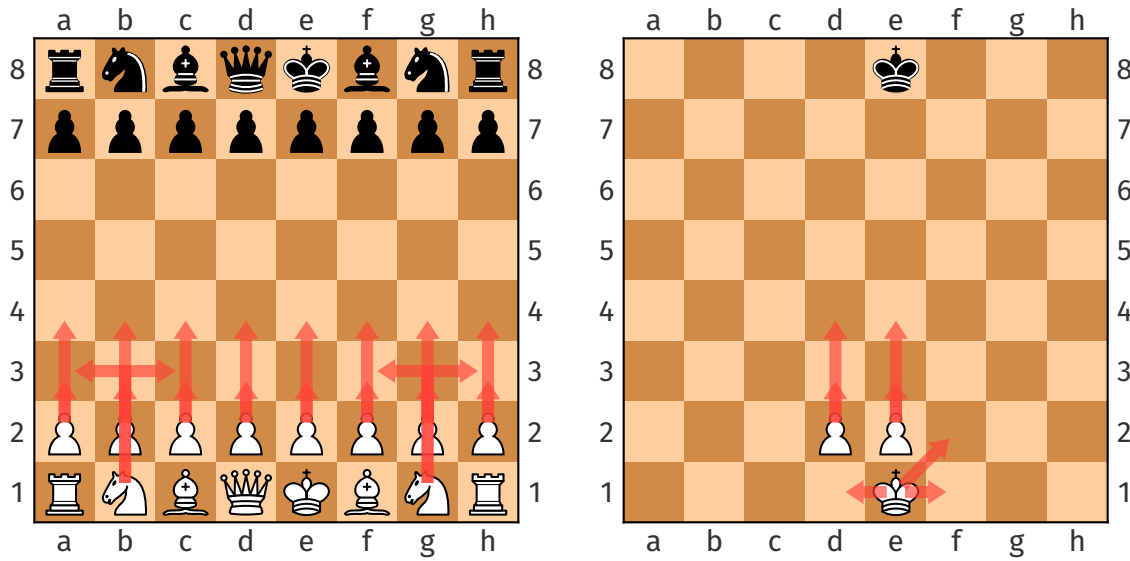


Figure 1: Red arrows show all the legal moves in both positions. As the number of legal moves changes depending on position, the model masks out illegal moves for to generate actions.

Behavior Cloning

For an agent to be successful in Chess it needs to master strategies and tactics. **Tactics** defines moves that achieve short term gains. This may be capturing opponent pieces or checkmating the opponent king. **Strategy** on the other hand defines moves with a long term benefit. This could be positioning a piece to put pressure on the opponents king, which may lead to a checkmate in 20-30 moves. See Figure 2 for example positions.

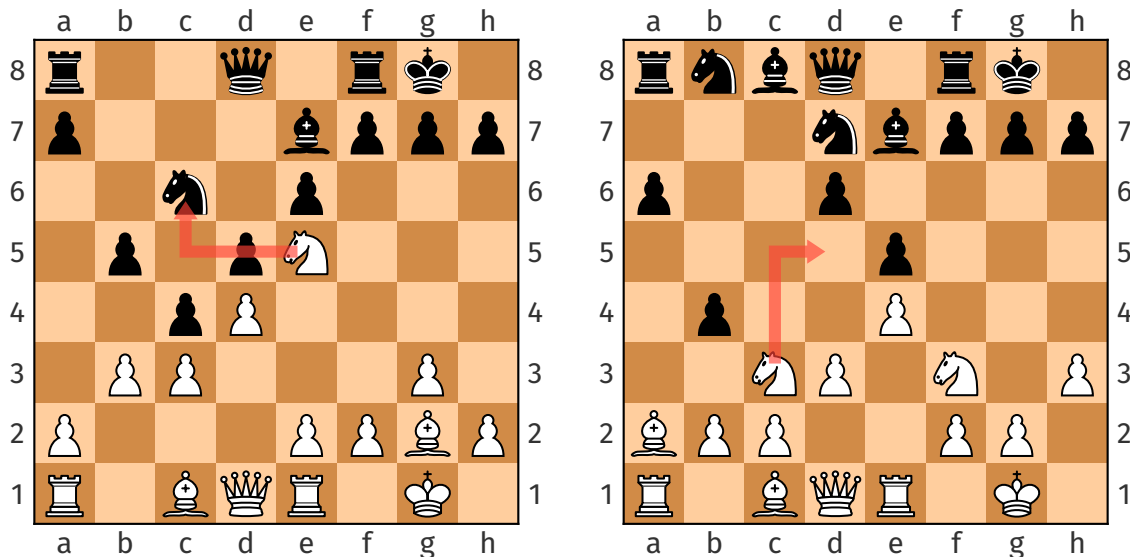


Figure 2: The position on the left has a tactical motif, where the black knight can be captured by white, as it is not defended. On the other hand the right position has a strategic motif, where the knight is attacked and has to retreat. There are multiple available squares, but d5 is the best option. There is no immediate gain, however in the long run the knight will be able to exert more pressure on the opponent from this square.

Puzzle Data

Players that want to improve their tactical perception can train on puzzles. A chess puzzle contains a position with a tactical motif (e.g. checkmate,...). A player then needs to play 4-5 moves to prove their understanding of the motif. [3]

There are many published datasets containing chess puzzles, such as These datasets are highly biased in that they solely contain tactical positions, underrepresenting strategic motifs. Furthermore, positions in such datasets always contain a tactical motif. The lack of negatives may lead the model to be tactically overconfident, leading to false positives (see Figure 3).

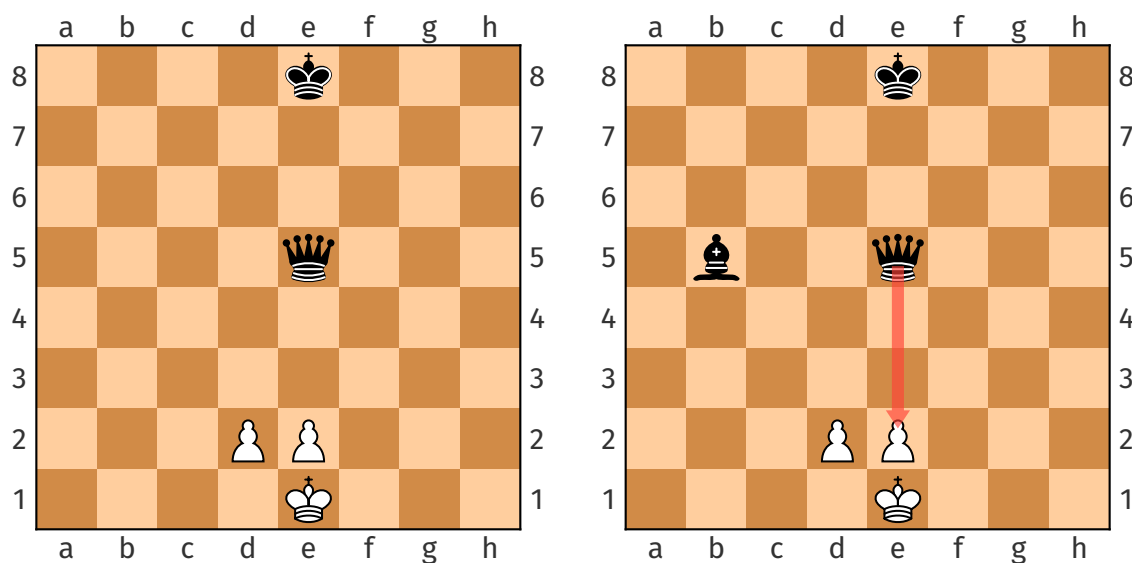


Figure 3: both positions are similar except for the black bishop on d5. In the position with the Bishop (right) black can win the game in one move by checkmating with the queen on e2. playing queen e2 in the other position will lose the game.

Grandmaster Data

Grandmaster games on the other hand are less biased than the puzzle dataset. They cover entire games, and therefore contain positions with tactical motives, and strategic motives [4]

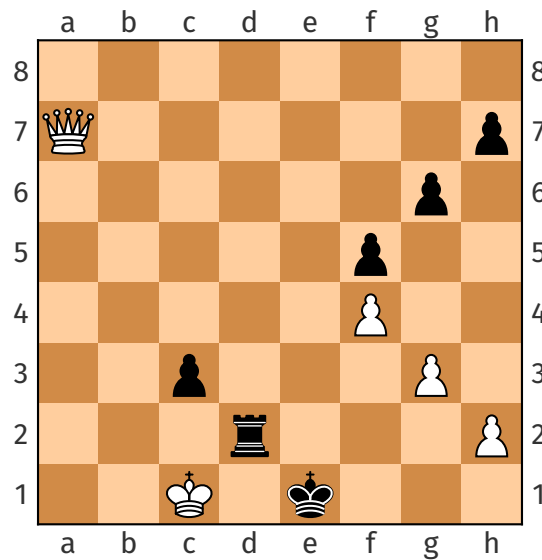


Figure 4: This is the last position of a game by the two former world champions Garry Kasparov and Veselin Topalov. Black resigns in this completely lost position. This is obvious to a grandmaster, in our case however, it will lead to an underrepresentation of checkmates, and will make it difficult for a model to convert completely winning positions.

Rewards Shaping

Findings

Bibliography

- [1] Tim Miller, “Reward Shaping.” [Online]. Available: <https://gibberblot.github.io/rl-notes/single-agent/reward-shaping.html>
- [2] “Imitation Learning, Stanford University.” [Online]. Available: https://web.stanford.edu/class/cs237b/pdfs/lecture/lecture_10111213.pdf
- [3] “Lichess chess puzzle dataset.” [Online]. Available: <https://www.kaggle.com/datasets/tianmin/lichess-chess-puzzle-dataset>
- [4] “All GM Chess Games on Chess.com.” [Online]. Available: <https://www.kaggle.com/datasets/dimitrioskourtikakis/gm-games-chesscom>